JOINS

✅ 1. What is a JOIN? (very simple)

A **JOIN** is used to:

combine rows from two (or more) tables using a related column.

In the data:

- orders are in one table
- customers are in another table

But...

➞ one order belongs to one customer
➞ so both tables share a key: customer_id

A JOIN connects them.

✅ 2. Why do we need JOIN?

Because real databases are **normalized**.

The data is split like this:

- order info → olist_orders_dataset
- customer info → olist_customers_dataset
- items inside each order → olist_order_items_dataset
- product info → olist_products_dataset
- payment info → olist_order_payments_dataset
- review info → olist_order_reviews_dataset

If joins didn't exist, we could never answer questions like:

show order id, customer city and total payment

That data lives in **3 different tables**.

✅ 3.The tables & how they connect (important)

From the files, the real keys are:

🔗 Main relationships

orders.customer_id = customers.customer_id

orders.order_id = order_items.order_id

order_items.product_id = products.product_id

order_items.seller_id = sellers.seller_id

orders.order_id = payments.order_id

orders.order_id = reviews.order_id

products.product_category_name = category_translation.product_category_name

This is the heart of all joins you will write.

✅ 4. What is an INNER JOIN?

✅ Definition

INNER JOIN returns only the rows that match in **both tables**.

✅ When do we use INNER JOIN?

When:

you only care about records that exist in both tables.

Example:

👉 orders that really have a customer
👉 orders that really have items
👉 orders that really have payments

✅ Syntax

SELECT columns FROM table1 INNER JOIN table2 ON condition;

(INNER is optional – you can write just JOIN)

✅ Example (your real data)

👉 Show order id and customer city

SELECT o.order_id, c.customer_city FROM olist_orders_dataset o INNER JOIN olist_customers_dataset c ON o.customer_id = c.customer_id;

✅ Why this works

We join:

orders.customer_id = customers.customer_id

Only orders that have a matching customer appear.

✅ 5. What is a LEFT JOIN?

✅ Definition

LEFT JOIN returns:

all rows from the LEFT table
and matching rows from the RIGHT table

If no match → right side becomes NULL.

✅ When do we use LEFT JOIN?

When:

you want everything from the main table
even if related data is missing.

✅ Syntax

SELECT columns FROM table1 LEFT JOIN table2 ON condition;

✅ Example (important case)

👉 Show all orders and their review score
(even if the order has no review yet)

SELECT o.order_id, r.review_score FROM olist_orders_dataset o LEFT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id;

✅ Why LEFT JOIN is correct here

Some orders have no reviews.

If you use INNER JOIN → those orders disappear.

LEFT JOIN keeps them.

✅ 6. What is a RIGHT JOIN?

✅ Definition

RIGHT JOIN returns:

all rows from the RIGHT table
and matching rows from the LEFT table

✅ When do we use RIGHT JOIN?

Almost never in real projects.

Because:

A RIGHT JOIN B

is the same as

B LEFT JOIN A

✅ Example (just for learning)

SELECT o.order_id, p.payment_value FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id;

This means:

👉 give me all payments even if an order row was missing.

💡 In practice, people prefer LEFT JOIN because it reads naturally.

✅ 7. Is there FULL JOIN in MySQL?

Short answer:

👉 ❌ MySQL does NOT support FULL JOIN directly.

But logically:

FULL JOIN means

give me everything from both tables
match when possible
keep unmatched rows from both sides

✅ MySQL trick using UNION

Example with orders and reviews:

SELECT o.order_id, r.review_score FROM olist_orders_dataset o LEFT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id UNION SELECT o.order_id, r.review_score FROM olist_orders_dataset o RIGHT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id;

This behaves like FULL OUTER JOIN.

✅ 8. Joining more than two tables (real-world case)

Let's do something very realistic.

👉 Show:

- order id
- customer city
- product category
- item price

SELECT o.order_id, c.customer_city, p.product_category_name, oi.price FROM olist_orders_dataset o JOIN olist_customers_dataset c ON o.customer_id = c.customer_id JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_products_dataset p ON oi.product_id = p.product_id;

This is how real analytics queries look.

You keep chaining joins.

✅ 9. Very important rule

👉 Always join using **keys**

Good joins in your data:

order_id

customer_id

product_id

seller_id

Bad joins:

city = city

state = state

Those are not unique and will explode your rows.

✅ 10. One tiny mental model (remember forever)

Think like this:

INNER JOIN

➞ intersection

LEFT JOIN

➞ keep everything from left

RIGHT JOIN

➞ keep everything from right

FULL JOIN

➞ keep everything

✅ 11. Quick cheat sheet

Join	Keeps left rows	Keeps right rows	Needs match?
INNER	❌	❌	✅
LEFT	✅	❌	no (right can be null)
RIGHT	❌	✅	no (left can be null)
FULL	✅	✅	no

Here are the **join types in MySQL** :

- **INNER JOIN**
- **LEFT JOIN**
- **RIGHT JOIN**
- **CROSS JOIN**
- **NATURAL JOIN**

Lets start our analysis :

🔹 **Orders + Customers (orders ↔ customers)**

Using:

olist_orders_dataset

olist_customers_dataset

(join on customer_id)

✅ **Business questions**

1. How many orders were placed by each **city**?
2. How many orders were placed by each **state**?
3. How many unique customers placed orders?
4. What is the number of orders per customer?
5. Which customers placed the **most orders**?

🔹 **Orders + Order Items (orders ↔ order_items)**

Using:

olist_orders_dataset

olist_order_items_dataset

(join on order_id)

✅ **Very important analytics questions**

6. How many items are there in each order?
7. What is the total sales value per order?
8. Which orders have the highest total item value?
9. What is the total freight value per order?
10. What is the average item price per order?

🔹 **Orders + Payments (orders ↔ payments)**

Using:

olist_orders_dataset

olist_order_payments_dataset

(join on order_id)

✅ **Revenue & payment analytics**

11. What is the total payment value per order?
12. Which payment type is used most often?
13. What is the total revenue by payment type?
14. What is the average payment value per order?

15. Which orders were paid in multiple installments?

🔹 **Orders + Reviews (orders ↔ reviews)**

Using:

olist_orders_dataset

olist_order_reviews_dataset

(join on order_id)

✅ **Customer satisfaction analytics**

16. What is the average review score per order?
17. How many orders received each review score?
18. Which orders received very low ratings?
19. What is the distribution of review scores?

🔹 **Orders + Items + Products**

(orders → order_items → products)

Using:

olist_orders_dataset

olist_order_items_dataset

olist_products_dataset

✅ **Product performance questions (very valuable)**

20. How many times was each product sold?
21. How many items were sold per product category?
22. What is the total revenue per product category?
23. Which products generate the highest revenue?
24. What is the average item price per category?

🔹 **Orders + Items + Sellers**

(orders → order_items → sellers)

Using:

olist_orders_dataset

olist_order_items_dataset

olist_sellers_dataset

✅ **Seller performance questions**

25. How many orders does each seller fulfill?
26. What is the total revenue per seller?
27. Which sellers sell the most items?
28. Which sellers generate the highest sales?

🔹 **Customers + Orders + Items + Products**

(full business view)

✅ **Very strong analytics questions**

29. Which customer cities generate the most revenue?
30. Which states buy which product categories the most?
31. What product categories are most popular in each state?

⚠️ **Very important rule**

All the above questions are perfect for **INNER JOIN** because:

👉 we only want records that actually exist in both tables
👉 missing relationships are not useful for these metrics

So logically:

✓ orders that really have customers
✓ orders that really have items
✓ items that really have products
✓ payments that really belong to orders

🧠 **One clean idea to remember**

For analytics like:

- sales
- revenue
- counts
- averages
- rankings
- performance

👉 **INNER JOIN is usually the default choice.**

---

1. Orders by city

SELECT c.customer_city, COUNT(*) AS total_orders FROM olist_orders_dataset o JOIN olist_customers_dataset c ON o.customer_id = c.customer_id GROUP BY c.customer_city;

→ joins orders to customers to get the city.

---

2. Orders by state

SELECT c.customer_state, COUNT(*) AS total_orders FROM olist_orders_dataset o JOIN olist_customers_dataset c ON o.customer_id = c.customer_id GROUP BY c.customer_state;

→ state is in customers table.

---

3. Unique customers who placed orders

SELECT COUNT(DISTINCT o.customer_id) FROM olist_orders_dataset o JOIN olist_customers_dataset c ON o.customer_id = c.customer_id;

→ only customers that really have orders.

---

4. Number of orders per customer

SELECT o.customer_id, COUNT(*) AS total_orders FROM olist_orders_dataset o JOIN olist_customers_dataset c ON o.customer_id = c.customer_id GROUP BY o.customer_id;

→ count orders per customer.

---

5. Customers with most orders

SELECT o.customer_id, COUNT(*) AS total_orders FROM olist_orders_dataset o JOIN olist_customers_dataset c ON o.customer_id = c.customer_id GROUP BY o.customer_id ORDER BY total_orders DESC;

→ ranking customers by orders.

---

6. Items per order

SELECT o.order_id, COUNT(*) AS items_count FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id GROUP BY o.order_id;

→ each row in order_items is one item.

---

7. Total item value per order

SELECT o.order_id, SUM(oi.price) AS total_value FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id GROUP BY o.order_id;

→ sum of item prices per order.

---

8. Highest value orders

SELECT o.order_id, SUM(oi.price) AS total_value FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id GROUP BY o.order_id ORDER BY total_value DESC;

→ same as above, ordered.

---

9. Freight per order

SELECT o.order_id, SUM(oi.freight_value) AS total_freight FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id GROUP BY o.order_id;

→ freight comes from order_items.

---

10. Average item price per order

SELECT o.order_id, AVG(oi.price) AS avg_price FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id GROUP BY o.order_id;

→ average per order.

---

11. Total payment per order

SELECT o.order_id, SUM(p.payment_value) AS total_payment FROM olist_orders_dataset o JOIN olist_order_payments_dataset p ON o.order_id = p.order_id GROUP BY o.order_id;

→ payments table holds values.

---

12. Most used payment type

SELECT p.payment_type, COUNT(*) AS cnt FROM olist_orders_dataset o JOIN olist_order_payments_dataset p ON o.order_id = p.order_id GROUP BY p.payment_type ORDER BY cnt DESC;

→ count by payment type.

---

13. Revenue by payment type

SELECT p.payment_type, SUM(p.payment_value) AS revenue FROM olist_orders_dataset o JOIN olist_order_payments_dataset p ON o.order_id = p.order_id GROUP BY p.payment_type;

→ sum per type.

---

14. Average payment per order

SELECT o.order_id, AVG(p.payment_value) AS avg_payment FROM olist_orders_dataset o JOIN olist_order_payments_dataset p ON o.order_id = p.order_id GROUP BY o.order_id;

→ average of payment rows per order.

---

15. Orders paid in installments

SELECT o.order_id, MAX(p.payment_installments) AS installments FROM olist_orders_dataset o JOIN olist_order_payments_dataset p ON o.order_id = p.order_id GROUP BY o.order_id HAVING MAX(p.payment_installments) > 1;

→ filter orders with more than 1 installment.

---

16. Average review score per order

SELECT o.order_id, AVG(r.review_score) AS avg_score FROM olist_orders_dataset o JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id GROUP BY o.order_id;

→ join orders and reviews.

---

17. Count of orders by review score

SELECT r.review_score, COUNT(*) AS total_orders FROM olist_orders_dataset o JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id GROUP BY r.review_score;

→ distribution of scores.

---

18. Orders with very low ratings

SELECT o.order_id, r.review_score FROM olist_orders_dataset o JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id WHERE r.review_score <= 2;

→ filter low scores.

---

19. Review score distribution

SELECT r.review_score, COUNT(*) AS cnt FROM olist_orders_dataset o JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id GROUP BY r.review_score;

→ same idea as #17.

---

20. Times each product sold

SELECT oi.product_id, COUNT(*) AS total_sold FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_products_dataset p ON oi.product_id = p.product_id GROUP BY oi.product_id;

→ count rows per product.

---

21. Items sold per category

SELECT p.product_category_name, COUNT(*) AS total_items FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_products_dataset p ON oi.product_id = p.product_id GROUP BY p.product_category_name;

→ category comes from products.

---

22. Revenue per category

SELECT p.product_category_name, SUM(oi.price) AS revenue FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_products_dataset p ON oi.product_id = p.product_id GROUP BY p.product_category_name;

→ sum prices by category.

---

23. Highest revenue products

SELECT oi.product_id, SUM(oi.price) AS revenue FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_products_dataset p ON oi.product_id = p.product_id GROUP BY oi.product_id ORDER BY revenue DESC;

→ rank products by revenue.

---

24. Average item price per category

SELECT p.product_category_name, AVG(oi.price) AS avg_price FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_products_dataset p ON oi.product_id = p.product_id GROUP BY p.product_category_name;

→ average price per category.

---

25. Orders per seller

SELECT oi.seller_id, COUNT(DISTINCT o.order_id) AS total_orders FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_sellers_dataset s ON oi.seller_id = s.seller_id GROUP BY oi.seller_id;

→ count orders handled by each seller.

---

26. Revenue per seller

SELECT oi.seller_id, SUM(oi.price) AS revenue FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_sellers_dataset s ON oi.seller_id = s.seller_id GROUP BY oi.seller_id;

→ seller sales value.

---

27. Items sold per seller

SELECT oi.seller_id, COUNT(*) AS total_items FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_sellers_dataset s ON oi.seller_id = s.seller_id GROUP BY oi.seller_id;

→ one row = one item.

---

28. Top sellers by revenue

SELECT oi.seller_id, SUM(oi.price) AS revenue FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_sellers_dataset s ON oi.seller_id = s.seller_id GROUP BY oi.seller_id ORDER BY revenue DESC;

→ ranking sellers.

---

29. Revenue by customer city

SELECT c.customer_city, SUM(oi.price) AS revenue FROM olist_orders_dataset o JOIN olist_customers_dataset c ON o.customer_id = c.customer_id JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id GROUP BY c.customer_city;

→ city + sales.

---

30. Most bought categories by state

SELECT c.customer_state, p.product_category_name, COUNT(*) AS total_items FROM olist_orders_dataset o JOIN olist_customers_dataset c ON o.customer_id = c.customer_id JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_products_dataset p ON oi.product_id = p.product_id GROUP BY c.customer_state, p.product_category_name;

→ state vs category demand.

---

31. Popular categories in each state

SELECT c.customer_state, p.product_category_name, COUNT(*) AS total_items FROM olist_orders_dataset o JOIN olist_customers_dataset c ON o.customer_id = c.customer_id JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_products_dataset p ON oi.product_id = p.product_id GROUP BY c.customer_state, p.product_category_name ORDER BY c.customer_state, total_items DESC;

→ same data, sorted for analysis.

LEFT JOIN

✅ What is LEFT JOIN?

LEFT JOIN returns all rows from the left table,
and only matching rows from the right table.
If there is no match → right-side columns become **NULL**.

✅ When do we use LEFT JOIN?

When your main table must **never lose rows**.

In your dataset, the main table is usually:

👉 olist_orders_dataset

Typical cases:

- some orders have **no reviews**
- some orders may have **no payments row yet**

✅ Syntax

SELECT … FROM A LEFT JOIN B ON A.key = B.key;

✅ Your first real example (very important)

👉 Show **all orders** and their review score (if any)

SELECT o.order_id, r.review_score FROM olist_orders_dataset o LEFT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id;

Why LEFT JOIN here?

Because:

👉 some orders do not have a review
👉 but we still want to see those orders

If you use INNER JOIN → those orders disappear.

✅ **Another very common case**

👉 Show all orders and their payment value

SELECT o.order_id, p.payment_value FROM olist_orders_dataset o LEFT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id;

✅ **Key idea (super important)**

FROM olist_orders_dataset o LEFT JOIN something

means:

👉 orders is your **master table**

You are saying:

"Keep all orders, no matter what."

✅ **How LEFT JOIN really behaves**

Think like this:

- order exists ✓
- related row exists ✓ → values shown
- related row missing ❌ → NULL shown

✅ **Very useful analytics pattern**

👉 Find orders **without** reviews

SELECT o.order_id FROM olist_orders_dataset o LEFT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id WHERE r.order_id IS NULL;

Why this is powerful?

LEFT JOIN + IS NULL
= find missing relationships

This is impossible with INNER JOIN.

✅ **One-line comparison**

- INNER JOIN → only matched rows
- LEFT JOIN → all left rows + matched right rows

---

1. Which **orders have no reviews** yet?
2. Which **orders have no payment records**?
3. How many **orders are missing reviews** by order status?
4. List **all orders** with their review score (show NULL if missing).
5. List **all orders** with their payment value (show NULL if missing).
6. Which **customers placed orders but never received a review**?
7. Which **orders have items but no recorded payment**?
8. Show **all orders and number of items**, including orders with **zero items**.
9. Which **products were never sold**?
10. Which **sellers have no sales**?

👉 In short:
Use **LEFT JOIN** when you want to **keep the main table rows and detect missing related data**.

They answer real business problems like:

👉 missing reviews
👉 missing payments
👉 unsold products
👉 inactive sellers
👉 data quality gaps

In real analytics, **LEFT JOIN questions are exactly how you find what is missing**, not just what exists.

1️⃣ Which orders have no reviews yet?

SELECT o.order_id FROM olist_orders_dataset o LEFT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id WHERE r.order_id IS NULL;

Why: keep all orders, then pick the ones where the review side is missing.

2️⃣ Which orders have no payment records?

SELECT o.order_id FROM olist_orders_dataset o LEFT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id WHERE p.order_id IS NULL;

Why: finds orders that do not have any matching payment row.

3️⃣ How many orders are missing reviews by order status?

SELECT o.order_status, COUNT(*) AS missing_reviews FROM olist_orders_dataset o LEFT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id WHERE r.order_id IS NULL GROUP BY o.order_status;

Why: after finding missing reviews, we group them by status.

4️⃣ List all orders with their review score (show NULL if missing)

SELECT o.order_id, r.review_score FROM olist_orders_dataset o LEFT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id;

Why: LEFT JOIN keeps every order even if no review exists.

5️⃣ List all orders with their payment value (show NULL if missing)

SELECT o.order_id, p.payment_value FROM olist_orders_dataset o LEFT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id;

Why: shows payment if it exists, otherwise NULL.

6️⃣ Which customers placed orders but those orders never received a review?

SELECT DISTINCT o.customer_id FROM olist_orders_dataset o LEFT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id WHERE r.order_id IS NULL;

Why: we keep orders, then return customers whose orders have no review.

7️⃣ Which orders have items but no recorded payment?

SELECT DISTINCT o.order_id FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id LEFT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id WHERE p.order_id IS NULL;

Why:
INNER JOIN guarantees the order has items,
LEFT JOIN checks if payment is missing.

8️⃣ Show all orders and number of items (including orders with zero items)

SELECT o.order_id, COUNT(oi.order_item_id) AS item_count FROM olist_orders_dataset o LEFT JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id GROUP BY o.order_id;

Why: LEFT JOIN keeps orders even when no item rows exist.

9️⃣ Which products were never sold?

SELECT p.product_id FROM olist_products_dataset p LEFT JOIN olist_order_items_dataset oi ON p.product_id = oi.product_id WHERE oi.product_id IS NULL;

Why: products with no matching order items are unsold.

🔟 Which sellers have no sales?

SELECT s.seller_id FROM olist_sellers_dataset s LEFT JOIN olist_order_items_dataset oi ON s.seller_id = oi.seller_id WHERE oi.seller_id IS NULL;

Why: sellers with no matching order items have no sales.

🧠 One simple pattern you should remember forever:

LEFT JOIN

...

WHERE right_table.key IS NULL

➡ means
"find what is missing."

RIGHT JOIN.

✅ What is RIGHT JOIN?

It keeps **all rows from the RIGHT table**, and only matching rows from the LEFT table.

✅ Syntax

SELECT ... FROM A RIGHT JOIN B ON A.id = B.id;

✅ Example (your dataset)

👉 Show **all payments**, even if an order row is missing.

SELECT o.order_id, p.payment_value FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id;

🧠 Important truth

In practice:

A RIGHT JOIN B

is the same as:

B LEFT JOIN A

👉 That's why analysts mostly use **LEFT JOIN** and almost never use RIGHT JOIN.

✅ What is RIGHT JOIN?

RIGHT JOIN keeps all rows from the RIGHT table
and matches rows from the LEFT table.

If there is no match →
👉 columns from the LEFT table become **NULL**

In one line:

RIGHT JOIN = keep the right table safe

✅ Syntax

SELECT … FROM A RIGHT JOIN B ON A.key = B.key;

This means:

👉 keep **all rows from B**

✅ Compare it with LEFT JOIN (very important)

LEFT JOIN:

FROM A LEFT JOIN B

➡ keep A

RIGHT JOIN:

FROM A RIGHT JOIN B

➡ keep B

That's the only difference.

✅ When do we really use RIGHT JOIN?

When your **main table is on the right side** and you want to keep all of it.

In real projects?

👉 almost never

Because people usually write the main table on the left and use LEFT JOIN.

But you still must understand it.

✅ Your real dataset example

👉 Show all payments even if the order row is missing

SELECT o.order_id, p.payment_value FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id;

Why is this RIGHT JOIN?

Because:

👉 we want to keep
olist_order_payments_dataset

That table is on the **right**.

So:

orders RIGHT JOIN payments

means:

👉 keep all payments

✅ What happens internally?

For each row in olist_order_payments_dataset:

- if a matching order exists → show order data
- if no order exists → o.order_id becomes NULL

✅ Detect missing relationships using RIGHT JOIN

Just like LEFT JOIN + IS NULL
we can do the same with RIGHT JOIN.

👉 Find payments that do NOT have a matching order

SELECT p.order_id, p.payment_value FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id WHERE o.order_id IS NULL;

Why this works

RIGHT JOIN keeps all payments.

If there is no order:

👉 o.order_id is NULL

So this query finds:

👉 orphan payment rows

✅ **Another realistic example**

👉 **Show all reviews even if the order row is missing**

SELECT o.order_id, r.review_score FROM olist_orders_dataset o RIGHT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id;

This keeps:

👉 all reviews

✅ **Find reviews without matching orders**

SELECT r.order_id, r.review_score FROM olist_orders_dataset o RIGHT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id WHERE o.order_id IS NULL;

✅ **Very important truth (memorise this)**

This:

SELECT ... FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id;

is exactly the same as:

SELECT ... FROM olist_order_payments_dataset p LEFT JOIN olist_orders_dataset o ON o.order_id = p.order_id;

So:

👉 RIGHT JOIN is just a reversed LEFT JOIN.

✅ **This is why analysts avoid RIGHT JOIN**

Because:

FROM payments p LEFT JOIN orders o

is:

- easier to read
- easier to reason about
- keeps the “main table” on the left

✅ **One small mental rule**

Look at the word:

👉 RIGHT JOIN

The table on the **right** is protected.

✅ **Very common mistake**

People write:

FROM orders o RIGHT JOIN payments p …

and then mentally think:

👉 orders are kept

❌ wrong

👉 payments are kept

✅ **Summary table**

Join	Which table is fully kept
LEFT JOIN	left table
RIGHT JOIN	right table

✅ **When would you actually use RIGHT JOIN?**

Mostly in:

👉 data quality checks
👉 auditing tables
👉 finding orphan records

For example:

- payments without orders
- reviews without orders

---

1. Which **payments do not have a matching order**?
2. Which **reviews do not have a matching order**?
3. Show **all payments**, even if the order record is missing.
4. Show **all reviews**, even if the order record is missing.
5. Are there any **payment records linked to non-existing orders**?
6. Are there any **review records linked to non-existing orders**?
7. How many **orphan payments** exist (payments without orders)?
8. How many **orphan reviews** exist (reviews without orders)?
9. List **payment values for records where the order is missing**.
10. List **review scores for records where the order is missing**.

👉 In short:
RIGHT JOIN in this dataset is mainly used to **check the right-side tables (payments, reviews) for orphan / unmatched records**.

1) Payments without matching order

SELECT p.* FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id WHERE o.order_id IS NULL;

---

2) Reviews without matching order

SELECT r.* FROM olist_orders_dataset o RIGHT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id WHERE o.order_id IS NULL;

---

3) All payments (even if order is missing)

SELECT o.order_id, p.payment_value, p.payment_type FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id;

---

4) All reviews (even if order is missing)

SELECT o.order_id, r.review_score FROM olist_orders_dataset o RIGHT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id;

---

5) Payment records linked to non-existing orders

SELECT p.order_id FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id WHERE o.order_id IS NULL;

---

6) Review records linked to non-existing orders

SELECT r.order_id FROM olist_orders_dataset o RIGHT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id WHERE o.order_id IS NULL;

---

7) How many orphan payments exist

SELECT COUNT(*) AS orphan_payments FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id WHERE o.order_id IS NULL;

---

8) How many orphan reviews exist

SELECT COUNT(*) AS orphan_reviews FROM olist_orders_dataset o RIGHT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id WHERE o.order_id IS NULL;

---

9) Payment values where the order is missing

SELECT p.order_id, p.payment_value FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id WHERE o.order_id IS NULL;

---

10) Review scores where the order is missing

SELECT r.order_id, r.review_score FROM olist_orders_dataset o RIGHT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id WHERE o.order_id IS NULL;

CROSS JOIN

✅ **CROSS JOIN – Definition**

A **CROSS JOIN** returns
👉 **every row from the first table combined with every row from the second table**.

In simple words:

it creates all possible combinations.

✅ **CROSS JOIN – Syntax**

SELECT … FROM table1 CROSS JOIN table2;

⚠️ Notice:
There is **NO ON condition** in a cross join.

✅ **What does it really do?**

If:

- table A has 5 rows
- table B has 10 rows

👉 result = **50 rows**

(5 × 10)

✅ When is CROSS JOIN useful in analysis?

Not for normal sales analysis.

It is mainly useful when:

👉 you want to generate combinations
👉 you want to compare every group with every other group
👉 you want a reference matrix

Typical real uses:

- all states × all product categories
- all sellers × all months
- all customers × all campaigns

✅ Let's use your dataset

We will use:

- olist_customers_dataset
- olist_products_dataset

Question 1

👉 Create all combinations of customer state and product category

SELECT DISTINCT c.customer_state, p.product_category_name FROM olist_customers_dataset c CROSS JOIN olist_products_dataset p;

Why this is useful

This creates a **state × category matrix**.

You can later LEFT JOIN real sales on top of this to find:

👉 which state–category combinations have zero sales.

Question 2

👉 **How many possible combinations of states and categories exist?**

SELECT COUNT(*) AS total_combinations FROM (SELECT DISTINCT customer_state FROM olist_customers_dataset) s CROSS JOIN (SELECT DISTINCT product_category_name FROM olist_products_dataset) c;

Why this is useful

This tells you:

👉 how big your analytical grid is
(for reporting, dashboards, completeness checks).

Question 3

👉 **Create all seller × category combinations**

SELECT DISTINCT s.seller_id, p.product_category_name FROM olist_sellers_dataset s CROSS JOIN olist_products_dataset p;

Why this is useful

You can use this to check later:

👉 which sellers never sold certain categories.

✅ **Very important warning**

Never do this by mistake:

FROM orders CROSS JOIN order_items

That would explode your data and destroy your results.

✅ **CROSS JOIN is NOT for relationships**

INNER / LEFT / RIGHT joins are for:

👉 linking tables

CROSS JOIN is for:

👉 generating combinations

✅ One-line summary

CROSS JOIN = combination generator, not data linker.

✅ Where CROSS JOIN is useful in real analytics

- building reporting templates
- building full grids for BI tools
- finding missing combinations
- completeness / coverage analysis

✅ Question 1

Which state × product category combinations have ZERO sales?

✅ Query

```
WITH grid AS ( SELECT DISTINCT c.customer_state, p.product_category_name FROM olist_customers_dataset c CROSS JOIN olist_products_dataset p ) SELECT g.customer_state, g.product_category_name FROM grid g LEFT JOIN ( SELECT c.customer_state, p.product_category_name FROM olist_orders_dataset o JOIN olist_customers_dataset c ON o.customer_id = c.customer_id JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_products_dataset p ON oi.product_id = p.product_id GROUP BY c.customer_state, p.product_category_name ) real_data ON g.customer_state = real_data.customer_state AND g.product_category_name = real_data.product_category_name WHERE real_data.product_category_name IS NULL;
```

✅ Why CROSS JOIN is needed

CROSS JOIN creates **all possible (state × category)** combinations.
Then we compare with real sales.

👉 This is how analysts find **missing markets**
(example: “this category was never sold in this state”).

✅ Question 2

Which sellers have never sold a specific product category?

✅ Query

```
WITH grid AS ( SELECT DISTINCT s.seller_id, p.product_category_name FROM olist_sellers_dataset s CROSS JOIN olist_products_dataset p ) SELECT g.seller_id, g.product_category_name FROM grid g LEFT JOIN ( SELECT oi.seller_id, p.product_category_name FROM olist_order_items_dataset oi JOIN olist_products_dataset p ON oi.product_id = p.product_id GROUP BY oi.seller_id, p.product_category_name ) real_data ON g.seller_id = real_data.seller_id AND g.product_category_name = real_data.product_category_name WHERE real_data.product_category_name IS NULL;
```

✅ Why CROSS JOIN is needed

We first create **all seller × category possibilities**.

Then we detect:

👉 which seller never sold that category

This is very useful for:

- seller performance analysis
- assortment / catalog planning

✅ Question 3

How many possible state × category combinations exist in our business?

✅ Query

```
SELECT COUNT(*) AS total_combinations FROM ( SELECT DISTINCT customer_state FROM olist_customers_dataset ) s CROSS JOIN ( SELECT DISTINCT product_category_name FROM olist_products_dataset ) c;
```

✅ Why CROSS JOIN is needed

This tells you the size of your **analysis grid**.

👉 Very useful when designing dashboards and coverage reports.

🧠 The real meaning of CROSS JOIN in analytics

CROSS JOIN is useful when you want to:

👉 create a **complete reference grid**
👉 and then find **what is missing in real data**

⚠️ **Important rule**

If your question is:

“connect orders to customers / products / payments”

❌ DO NOT use CROSS JOIN.

If your question is:

“show me all possible combinations and find gaps”

✅ that is exactly when CROSS JOIN is correct.

CROSS JOINS

✅ Question 1

Create all possible customer_state × product_category combinations

SELECT DISTINCT c.customer_state, p.product_category_name FROM olist_customers_dataset c CROSS JOIN olist_products_dataset p;

Explanation:
CROSS JOIN generates every state with every product category.

✅ Question 2

Create all possible seller × product_category combinations

SELECT DISTINCT s.seller_id, p.product_category_name FROM olist_sellers_dataset s CROSS JOIN olist_products_dataset p;

Explanation:
Every seller is paired with every category.

✅ Question 3

Create all possible customer × seller combinations

SELECT DISTINCT c.customer_id, s.seller_id FROM olist_customers_dataset c CROSS JOIN olist_sellers_dataset s;

Explanation:
Creates a full matrix of customers and sellers (useful for coverage or matching analysis).

🧠 **Very important note (honest answer)**

With **only CROSS JOIN**, you can:

👉 generate combinations
👉 but you cannot check real sales or gaps

Because checking gaps **always requires another join**.

So remember:

CROSS JOIN alone = combination generator only.

That is its real and only analytical role.

OUTER JOIN

MySQL does NOT support FULL OUTER JOIN **directly.**
So we simulate it using:

LEFT JOIN

UNION

RIGHT JOIN

✅ **Definition (FULL OUTER JOIN)**

A **FULL OUTER JOIN** returns:

👉 all rows from the left table
👉 all rows from the right table
👉 and matches them where possible.

Unmatched rows from either side are kept.

✅ **Syntax in MySQL (the only correct way)**

SELECT ... FROM A LEFT JOIN B ON ... UNION SELECT ... FROM A RIGHT JOIN B ON ...

✅ **When is FULL OUTER JOIN useful in analysis?**

When you want to:

👉 compare two related tables
👉 and also detect missing records on both sides

Main use:

- data quality checks
- reconciliation
- auditing pipelines

Now let's do real questions on your dataset.

🔹 **Question 1**

👉 **Show all orders and all reviews (including:**

orders without reviews AND reviews without orders)

✅ **Query**

SELECT o.order_id, r.review_score FROM olist_orders_dataset o LEFT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id UNION SELECT o.order_id, r.review_score FROM olist_orders_dataset o RIGHT JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id;

✅ **Explanation**

LEFT JOIN part:

👉 keeps all orders

RIGHT JOIN part:

👉 keeps all reviews

UNION:

👉 merges both

So the final result contains:

- orders that have reviews
- orders that do NOT have reviews
- reviews that do NOT have matching orders

This is a true FULL OUTER JOIN.

✅ **Why this is useful**

To audit:

👉 is every review really linked to an order?
👉 which orders still have no reviews?

🔹 **Question 2**

👉 **Show all orders and all payments (even if either side is missing)**

✅ **Query**

SELECT o.order_id, p.payment_value FROM olist_orders_dataset o LEFT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id UNION SELECT o.order_id, p.payment_value FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id;

✅ **Explanation**

This returns:

- orders with payments
- orders without payments
- payments without orders

✅ **Why this is useful**

In real analytics this is used to detect:

👉 unpaid orders
👉 orphan payments
👉 broken ETL loads

🔹 **Question 3**

👉 **Find all order IDs that are problematic**

(either missing payment or missing order)

✅ **Query**

SELECT o.order_id, p.order_id AS payment_order_id FROM olist_orders_dataset o LEFT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id UNION SELECT o.order_id, p.order_id AS payment_order_id FROM olist_orders_dataset o RIGHT JOIN olist_order_payments_dataset p ON o.order_id = p.order_id;

✅ **Explanation**

After this result:

- rows where payment_order_id is NULL
  → order exists but payment is missing
- rows where order_id is NULL
  → payment exists but order is missing

🧠 **Very important rule**

In MySQL:

❌ this is invalid:

FULL OUTER JOIN

✓ correct way is always:

LEFT JOIN

UNION

RIGHT JOIN

✅ One-line summary

FULL OUTER JOIN is mainly used for data validation and reconciliation — not for normal sales analysis.

NATURAL JOIN

✅ What is NATURAL JOIN?

A **NATURAL JOIN** automatically joins two tables
using columns that have the **same name in both tables**.

You do NOT write the ON condition.

✅ Syntax

SELECT ... FROM table1 NATURAL JOIN table2;

No ON clause.

✅ How it works internally

If both tables have a column with the same name, for example:

customer_id

order_id

product_id

MySQL automatically joins using those columns.

⚠️ Very Important Warning

NATURAL JOIN:

- joins on **ALL same-named columns**
- you have no control

- can break easily if schema changes

That's why in real analytics:

👉 professionals rarely use NATURAL JOIN

✅ **Let's use your dataset**

Example 1

👉 **Join orders and customers using NATURAL JOIN**

Both tables share:

customer_id

SELECT * FROM olist_orders_dataset NATURAL JOIN olist_customers_dataset;

What happens?

MySQL automatically does:

ON olist_orders_dataset.customer_id = olist_customers_dataset.customer_id

---

✅ **Example 2**

👉 **Join orders and order_items**

Both share:

order_id

SELECT * FROM olist_orders_dataset NATURAL JOIN olist_order_items_dataset;

Automatically joins using order_id.

✅ **When is NATURAL JOIN useful?**

Very rarely.

It is useful only when:

- you are 100% sure column names match correctly
- your schema is clean and stable
- quick ad-hoc exploration

❌ When should you NOT use NATURAL JOIN?

In production analytics.

Because:

If tomorrow someone adds another column with the same name
👉 your query behavior changes automatically.

That is dangerous.

✅ Professional rule

Instead of:

NATURAL JOIN

Use:

JOIN … ON …

Because:

✓ more readable
✓ more controlled
✓ safer

🧠 Very important difference

INNER JOIN:

JOIN table2 ON table1.id = table2.id

You choose the column.

NATURAL JOIN:

NATURAL JOIN table2

MySQL chooses the column.

✅ One-line summary

NATURAL JOIN is automatic joining on same column names — convenient but risky.

Since NATURAL JOIN works only when **column names are the same**, in your Olist dataset we can use it where columns like:

- customer_id
- order_id
- product_id
- seller_id

exist in both tables.

✅ 1️⃣ Question

Show all orders with customer city and state

SELECT order_id, customer_id, customer_city, customer_state FROM olist_orders_dataset NATURAL JOIN olist_customers_dataset;

Why this works

Both tables share customer_id, so NATURAL JOIN automatically joins on it.

✅ 2️⃣ Question

Show all orders and their item prices

SELECT order_id, product_id, price FROM olist_orders_dataset NATURAL JOIN olist_order_items_dataset;

Why this works

Both tables share order_id.

✅ 3️⃣ Question

Show order items with product category

SELECT order_id, product_id, product_category_name FROM olist_order_items_dataset NATURAL JOIN olist_products_dataset;

Why this works

Both tables share product_id.

✅ 4️⃣ Question

Show order payments with order status

SELECT order_id, payment_type, payment_value, order_status FROM olist_orders_dataset NATURAL JOIN olist_order_payments_dataset;

Why this works

Both share order_id.

🧠 Important Reminder

NATURAL JOIN is fine for:

👉 quick exploration
👉 learning

But in real analytics, we prefer:

JOIN … ON …

Because it's safer and clearer.

SELF JOIN

✅ What is SELF JOIN?

A **SELF JOIN** is when a table is joined with itself.

It is NOT a new join type.

It is just:

table JOIN table

But we use **aliases** to treat them like two different tables.

✅ Syntax

SELECT … FROM table A JOIN table B ON A.column = B.column;

⚠️ Important:
We must use aliases like A and B.

🧠 **When is SELF JOIN useful in analytics?**

Use SELF JOIN when:

👉 you want to compare rows inside the same table
👉 you want to find duplicates
👉 you want to compare customers with other customers
👉 you want to compare sellers with other sellers

Now let's use your Olist dataset.

✅ 1️⃣ **Question**

Find customers who live in the same city

SELECT c1.customer_id AS customer_1, c2.customer_id AS customer_2, c1.customer_city FROM olist_customers_dataset c1 JOIN olist_customers_dataset c2 ON c1.customer_city = c2.customer_city AND c1.customer_id <> c2.customer_id;

Why this is useful

👉 Find customers from the same city
Useful for:

- regional marketing
- clustering analysis

✅ 2️⃣ **Question**

Find sellers in the same state

SELECT s1.seller_id AS seller_1, s2.seller_id AS seller_2, s1.seller_state FROM olist_sellers_dataset s1 JOIN olist_sellers_dataset s2 ON s1.seller_state = s2.seller_state AND s1.seller_id <> s2.seller_id;

Why useful?

👉 Market competition analysis
Which sellers operate in the same state?

✅ 3️⃣ Question

Find duplicate product categories

SELECT p1.product_id, p2.product_id, p1.product_category_name FROM olist_products_dataset p1 JOIN olist_products_dataset p2 ON p1.product_category_name = p2.product_category_name AND p1.product_id <> p2.product_id;

Why useful?

👉 Identify products inside same category
Useful for:

- competitor product comparison
- pricing analysis

✅ 4️⃣ Question

Find customers in same ZIP code prefix

SELECT c1.customer_id, c2.customer_id, c1.customer_zip_code_prefix FROM olist_customers_dataset c1 JOIN olist_customers_dataset c2 ON c1.customer_zip_code_prefix = c2.customer_zip_code_prefix AND c1.customer_id <> c2.customer_id;

Why useful?

👉 Local market density analysis

🧠 Very important rule

When using SELF JOIN:

Always prevent matching the same row:

AND table1.id <> table2.id

Otherwise you match row with itself.

🔥 **Where SELF JOIN is very powerful in real world?**

- finding duplicates
- fraud detection
- identifying similar customers
- hierarchical relationships (manager → employee)
- comparing prices inside same category

One-line summary

SELF JOIN = compare rows within the same table.

You've now learned:

- INNER
- LEFT
- RIGHT
- CROSS
- FULL OUTER (via UNION)
- NATURAL
- SELF

🎯 **MINI PROJECT: E-Commerce Business Performance Report**

Management wants:

"Give us a performance overview of customers, products, sellers, and satisfaction."

We'll solve this in **4 tasks** (real business-level questions).

🧩 **Task 1 — Revenue by State**

❓ **Business Question**

Which states generate the highest revenue?

✅ Query

SELECT c.customer_state, SUM(oi.price) AS total_revenue FROM olist_orders_dataset o JOIN olist_customers_dataset c ON o.customer_id = c.customer_id JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id GROUP BY c.customer_state ORDER BY total_revenue DESC;

🎯 What this tells management

- Top revenue states
- Where marketing budget should go
- Strong vs weak regions

🧩 Task 2 — Top 10 Product Categories by Revenue

❓ Business Question

Which categories make the most money?

✅ Query

SELECT p.product_category_name, SUM(oi.price) AS revenue FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id JOIN olist_products_dataset p ON oi.product_id = p.product_id GROUP BY p.product_category_name ORDER BY revenue DESC LIMIT 10;

🎯 Why important?

- Identify core business categories
- Focus inventory planning
- Plan promotions

🧩 Task 3 — Seller Performance Ranking

❓ Business Question

Which sellers generate the most revenue?

✅ Query

SELECT oi.seller_id, SUM(oi.price) AS revenue FROM olist_orders_dataset o JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id GROUP BY oi.seller_id ORDER BY revenue DESC LIMIT 10;

🎯 **Why important?**

- Identify top sellers
- Reward high performers
- Detect dependency risk

🧩 **Task 4 — Revenue vs Review Score**

❓ **Business Question**

Do higher review scores correlate with revenue?

✅ **Query**

SELECT r.review_score, SUM(oi.price) AS revenue FROM olist_orders_dataset o JOIN olist_order_reviews_dataset r ON o.order_id = r.order_id JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id GROUP BY r.review_score ORDER BY r.review_score;

🎯 **Why important?**

- See if bad reviews impact sales
- Check customer satisfaction effect

🧠 **What You Just Did**

You combined:

- INNER JOIN
- GROUP BY
- SUM()
- ORDER BY
- LIMIT

This is real-world analytics.